UNIVERSIDAD MAYOR DE SAN ANDRÉS

Facultad de Ciencias Puras y Naturales

Carrera de Informática

Sistema de certificación académica mediante Blockchain en la nube

Informe integral de funcionamiento e implementación

Manual técnico y de usuario con instalación y validación

Datos académicos	Información
Materia	Computación en la Nube
Docente	Lic. José María Tapia Baltazar
Equipo	Grupo 1
Versión del sistema	Prototipo académico local 1.0.0
Fecha de documentación	22 de octubre de 2026
Lugar	La Paz, Bolivia

Integrantes del equipo

Enriquez Aduviri Vanesa Alejandra

Chambi Loza Angela Miriam

Flores Mamani Magali

Marino Flores Harol Jeremias

Maldonado Huanca Bladimir Carlos

Prototipo académico - Sin validez oficial

Este documento contiene las credenciales de demostración del entorno local y reúne el diseño aplicado, la operación y las evidencias del sistema desarrollado.

Resumen y organización del documento

El sistema permite registrar estudiantes, emitir certificados académicos en PDF, registrar sus huellas criptográficas en un contrato inteligente, verificar su autenticidad y vigencia y revocarlos conservando el historial. La emisión, la consulta pública y la revocación funcionan sobre una red EVM real de Hardhat ejecutada localmente.

La implementación conserva la arquitectura modular y la separación de datos del Capítulo IV del informe del proyecto. React presenta la interfaz; Express concentra la lógica y los permisos; SQLite conserva los datos académicos; el almacenamiento privado custodia los PDF; y Solidity registra los hashes y el estado de revocación. La cadena no contiene nombres ni documentos de identidad.

La validación ejecutada aprobó ocho pruebas del contrato y quince casos de integración. Además, se utilizó el navegador para registrar un estudiante ficticio, emitir y descargar certificados, verificar un documento y revocarlo. Se comprobó la recuperación de la información al reiniciar los procesos. La instalación entregada es local; el despliegue en servicios de nube queda como una etapa posterior.

Contenido y ruta de lectura

Parte	Contenido	Página
Capítulo I	Contexto, problema, objetivos y alcance del prototipo	3
Capítulo II	Tecnologías y conceptos utilizados	4
Capítulo III	Actores, requisitos y funciones del sistema	5
Capítulo IV	Arquitectura, datos, contrato, procesos y seguridad	6
Capítulo V	Implementación, instalación, credenciales y manual de uso	10
Capítulo VI	Pruebas, resultados y evidencias de funcionamiento	16
Capítulo VII	Conclusiones, límites y evolución hacia la nube	21
Anexos	API, solución de problemas, guion de exposición y referencias	22

Cómo utilizar este informe

Para comprender el proyecto, comenzar por los capítulos I a IV. Para instalarlo y realizar una demostración, utilizar el capítulo V. Para sustentar qué se probó realmente, consultar el capítulo VI. Las credenciales iniciales se incluyen en la sección de instalación y acceso, sin necesidad de abrir otros documentos.

Capítulo I Aspectos generales del proyecto

1 1 Contexto y problema

La certificación académica requiere acreditar que una institución emitió un documento y que su contenido no ha sido alterado. Un archivo PDF puede copiarse o modificarse, por lo que la apariencia visual del certificado por sí sola no permite confirmar su autenticidad. La consulta manual al emisor también introduce demoras y dependencia de la disponibilidad de una oficina o de un registro interno.

El proyecto plantea una aplicación que conecta la gestión académica con un registro criptográfico verificable. La base de datos conserva la información necesaria para administrar los documentos, mientras que Blockchain permite consultar si la huella de un certificado fue registrada y si posteriormente se revocó. El prototipo permite observar este proceso de principio a fin en una exposición universitaria.

1 2 Objetivo general

Implementar un prototipo web funcional de certificación académica que permita emitir, registrar, verificar y revocar certificados mediante un contrato inteligente, con persistencia de datos y una interfaz institucional accesible.

1 3 Objetivos específicos

- Administrar usuarios y estudiantes mediante autenticación y permisos por rol.
- Generar certificados PDF con identificador único, código QR y leyenda de prototipo.
- Calcular SHA-256 de los datos y del archivo y registrar ambos hashes en una cadena EVM.
- Permitir la verificación pública por código, enlace QR y archivo PDF.
- Registrar revocaciones definitivas y conservar las transacciones y la bitácora.
- Automatizar la instalación y comprobar los procesos mediante pruebas reproducibles.

1 4 Alcance y adaptación del informe

La versión implementada utiliza una institución emisora y una wallet institucional administrada por el backend. Funciona inicialmente en Windows con SQLite y almacenamiento local privado. No incorpora servicios de pago ni requiere que el usuario del navegador conecte una billetera.

El informe original propone infraestructura AWS, PostgreSQL y acceso a una red externa. En el prototipo se sustituyen por recursos locales para facilitar la instalación, sin sustituir Blockchain por registros simulados. Los documentos emitidos tienen fines académicos y no representan certificados oficialmente autorizados por la universidad.

Capítulo II Tecnologías y conceptos utilizados

Las tecnologías se organizan por responsabilidad. El navegador no accede directamente a SQLite ni a las claves de firma; utiliza solicitudes HTTP al backend. Las versiones siguientes corresponden a los archivos de dependencias del proyecto entregado.

Tecnología	Versión	Uso en el sistema
React	19.1.1	Componentes y estado de la interfaz
Vite	6.4.5	Servidor de desarrollo y compilación del frontend
Tailwind CSS	3.4.17	Utilidades de estilo junto con CSS propio
React Router DOM	7.18.4	Navegación pública y protegida
Lucide React	0.468.0	Iconos de la interfaz
Node.js	26.8.1 probado	Ejecución del backend y scripts
Express	5.1.0	API REST y middleware
SQLite	Integrado en Node.js	Persistencia en archivo local
ethers	6.17.0	Firma y comunicación JSON-RPC
Hardhat	2.26.3	Red local, compilación y pruebas EVM
Solidity y solc	0.8.28	Contrato y compilador local
OpenZeppelin	5.4.0	Control de acceso del contrato
pdf-lib y qrcode	1.17.1 y 1.5.4	PDF institucional e imagen QR
JWT y bcryptjs	9.0.2 y 3.0.2	Sesiones y hash de contraseñas

2 1 Herramientas complementarias

Fetch conecta React con la API. Zod valida entradas en el backend; Helmet añade cabeceras de seguridad; express-rate-limit controla la frecuencia de solicitudes; Multer recibe archivos; csv-parse interpreta lotes; dotenv carga la configuración; y el módulo crypto de Node.js genera UUID, salt y hashes. Las pruebas utilizan Hardhat, assert y el runner de pruebas de Node.js; jsQR comprueba el QR incrustado en el PDF.

2 2 Conceptos esenciales

Un hash es una huella calculada a partir de datos: cambiar los datos produce otra huella. Un salt es un valor aleatorio que se incorpora al cálculo para evitar que datos conocidos permitan reconstruir fácilmente el hash. Un contrato inteligente es código que se ejecuta en la EVM y aplica las reglas de registro y revocación. Una transacción modifica ese registro y genera un recibo verificable.

Capítulo III Análisis funcional del sistema

3 1 Actores y responsabilidades

Actor	Operaciones permitidas
Administrador	Gestionar y autorizar usuarios; consultar todos los certificados; ver estadísticas y bitácora; revocar; reconciliar estados.
Emisor institucional	Registrar y editar estudiantes; emitir certificados individuales o por CSV; consultar y descargar sus certificados; revocar los propios.
Verificador público	Consultar código o enlace QR y subir un PDF para verificar integridad y vigencia sin iniciar sesión.

El estudiante es titular de los datos del certificado, pero esta versión no incluye un portal de autoservicio para estudiantes. La verificación pública no crea una cuenta y no otorga acceso a la administración.

3 2 Funciones principales

- Autenticación institucional con JWT y rutas protegidas.
- Gestión de estudiantes con documento de identidad único.
- Dashboard calculado a partir de registros de SQLite.
- Emisión con UUID, salt, dos hashes SHA-256 y confirmación de transacción.
- Generación y descarga autorizada del PDF con QR.
- Consulta pública del estado de vigencia e integridad.
- Revocación con justificación y conservación del historial.
- Administración de usuarios y bitácora de acciones.

3 3 Información registrada

El estudiante tiene nombres, apellidos, documento, carrera y correo opcional. Para emitir, se selecciona un estudiante y se indica tipo de certificado, grado o reconocimiento, fecha e institución. El código del certificado se genera automáticamente a partir de un UUID para evitar colisiones y errores de captura.

3 4 Criterio de aceptación

Una emisión se considera completada cuando existe un PDF privado, sus hashes han sido registrados y la transacción tiene recibo exitoso. Un registro pendiente o con error no habilita una descarga como certificado emitido. La verificación pública debe consultar correctamente el contrato; una interrupción de Blockchain produce un mensaje de error, nunca una confirmación de validez.

Capítulo IV Arquitectura del sistema

4 1 Arquitectura general híbrida

Se implementó un monolito modular: una API, una base de datos y un contrato. La arquitectura es híbrida porque divide la custodia de información entre datos privados fuera de la cadena y huellas criptográficas dentro de ella. En esta entrega, la parte que podría alojarse en la nube se ejecuta en la computadora local.

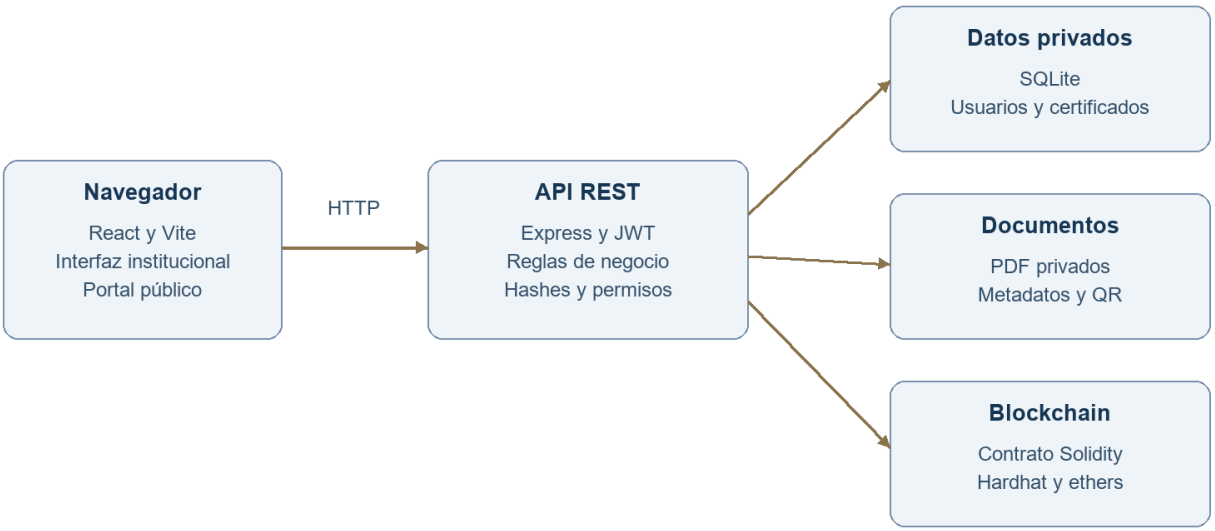


Figura 1 Arquitectura de la implementación local

4 1 1 Responsabilidad de cada capa

La presentación utiliza React, rutas de navegación y formularios. Vite sirve el frontend y reenvía las solicitudes /api al backend. Express aplica autenticación, autorización, validación y reglas de emisión; también coordina SQLite, la generación de archivos y la comunicación con la cadena.

SQLite conserva los usuarios, estudiantes, certificados, transacciones y acciones de auditoría. Los PDF se guardan en una carpeta privada. El contrato AcademicCertRegistry aplica los permisos de las wallets y conserva los hashes y la revocación. Hardhat ejecuta la EVM y expone el servicio JSON-RPC al que se conecta ethers.

4 1 2 Correspondencia con el diseño original

Diseño del informe	Implementación entregada
React y backend modular	React con Vite y Express modular
PostgreSQL con Prisma	SQLite con consultas parametrizadas
Bucket S3 privado	Carpeta runtime/pdf no pública
RPC externo y Sepolia	Nodo Hardhat local con chain ID 31337
Secretos gestionados en nube	Variables privadas del backend en .env

4 2 Diseño y persistencia de los datos

Los datos personales permanecen fuera de Blockchain. La base SQLite utiliza claves primarias, relaciones mediante claves foráneas y restricciones únicas. La migración 001.sql crea el esquema de manera idempotente y se registra una versión de migración.

Tabla	Contenido y relación
usuarios	Identificador, nombre, correo único, contraseña con bcrypt, rol y estado activo. Se relaciona con certificados y bitácora.
estudiantes	Identificador, nombres, apellidos, documento único, carrera, correo y fecha de registro. Un estudiante puede recibir varios certificados.
certificados	Estudiante y emisor asociados; copia de datos; UUID, código, salt, hash de datos, hash PDF, ruta privada, estado, transacciones y datos de revocación.
transacciones	Certificado asociado, tipo de operación, hash de transacción, estado y fecha. Conserva el historial de envíos.
bitacora	Usuario, acción, detalle y fecha. Registra emisiones, revocaciones y acciones administrativas.

4 2 1 Copia de los datos académicos

Durante la emisión se toma una copia de los datos del estudiante y se guarda en el campo datos del certificado. Editar posteriormente el registro del estudiante no cambia esa copia ni modifica el PDF emitido. Esta separación permite conservar el contenido que dio origen al hash registrado.

4 2 2 Información dentro de la cadena

Cada registro del contrato se identifica por certHash y contiene la dirección emisora, la fecha del bloque de emisión, la fecha de revocación y pdfHash. El valor cero en la fecha de revocación representa un registro vigente. Los eventos permiten observar las operaciones de emisión y revocación.

4 2 3 Archivos privados y respaldo

Los archivos se almacenan en runtime/pdf y se entregan a través de una ruta autenticada. No se publican como recursos estáticos. El archivo principal de SQLite es runtime/academic.sqlite. Para conservar una demostración completa deben respaldarse runtime y .env con el sistema detenido, incluyendo deployment.json y chain.jsonl.

4 3 Hashes y contrato inteligente

4 3 1 Construcción de la huella de datos

Cada certificado recibe un UUID y un salt generado con 16 bytes aleatorios. Se construye un objeto que incluye identificador, código, nombres, apellidos, documento, carrera, tipo, grado, fecha, institución y salt. Sus claves se ordenan alfabéticamente y el objeto se serializa como JSON sin espacios. Sobre sus bytes UTF-8 se calcula SHA-256.

La representación canónica evita que un cambio en el orden de las claves produzca una huella diferente para los mismos valores. El salt pertenece al almacenamiento privado y no se envía como un dato independiente al contrato.

4 3 2 Huella del archivo PDF

El PDF se genera antes de enviarse la transacción. Su SHA-256 se calcula sobre el archivo completo, no sobre el texto visible. El contrato registra ambos hashes en una misma operación. De esta forma se vincula la información académica con los bytes exactos del documento entregado.

4 3 3 Funciones y permisos

Elemento	Comportamiento implementado
issue certHash y pdfHash	Solo ISSUER_ROLE. Rechaza hashes vacíos y registros duplicados. Guarda emisor y fecha y emite CertificateIssued.
verify certHash	Consulta pública de existencia, revocación, emisor, fecha y hash del PDF. No modifica el estado.
revoke certHash	Permite al emisor original autorizado o al administrador del contrato revocar. Rechaza inexistentes y revocaciones repetidas.
DEFAULT_ADMIN_ROLE	Gestiona autorizaciones de wallets mediante AccessControl.
ISSUER_ROLE	Autoriza la wallet que registra certificados.
Eventos	CertificateIssued y CertificateRevoked conservan evidencia de las operaciones.

El diseño original contempla issue con un hash. La implementación lo amplía a dos hashes para comprobar el PDF contra la cadena. Los roles del contrato son distintos de los roles de usuarios web: los emisores institucionales de la aplicación comparten la wallet emisora configurada en el backend.

4 4 Procesos estados y seguridad

4 4 1 Emisión de un certificado

1. El backend autentica al usuario, valida los campos y busca al estudiante.

2. Genera UUID, código, salt, representación canónica y hash de datos.

3. Inserta el registro pendiente en SQLite; genera el PDF con QR y obtiene su hash.

4. Firma y envía issue con ambos hashes. Guarda la evidencia de envío y espera el recibo.

5. Si el recibo es exitoso, marca emitido y registra la bitácora. Si falla, conserva error para revisión y recuperación.

4 4 2 Verificación y revocación

La consulta por código o QR recupera el certificado, recalcula la huella de sus datos y consulta verify en el contrato. La consulta por PDF añade la lectura del UUID desde metadatos y la comparación SHA-256 del archivo con pdfHash. La revocación exige una justificación, aplica los permisos y espera confirmación de la operación en cadena antes de actualizar el estado local.

Estado	Significado
pendiente	Emisión en proceso, sin descarga habilitada.
emitido	Emisión confirmada y documento disponible.
revocado	Certificado revocado, conservado para consulta.
error	Proceso incompleto que requiere revisión.

4 5 Interfaz y seguridad de acceso

La interfaz dispone de menú lateral, barra de sesión, formularios, buscadores, filtros, confirmaciones y notificaciones. El portal público adapta la consulta al uso en móviles. Los permisos del servidor protegen las acciones aunque una persona intente acceder a una ruta directamente.

4 6 Seguridad y privacidad

Las contraseñas se guardan con bcrypt y costo 12. Los JWT vencen en una hora; cada petición privada verifica que el usuario siga activo. Zod valida entradas, SQLite utiliza parámetros, Helmet añade cabeceras y se limita la frecuencia de solicitudes. Las claves de firma permanecen en el backend y los PDF fuera de public.

La información pública se reduce al titular, datos académicos, institución, fecha y evidencia criptográfica. No se muestran documento de identidad, correo, salt o ruta del archivo. El navegador elimina su token al cerrar sesión. El prototipo utiliza HTTP local; un despliegue público requeriría HTTPS y una gestión de secretos apropiada.

Capítulo V Implementación del proyecto

5 1 Organización del código

Archivo o carpeta	Responsabilidad
frontend/main.jsx	Pantallas, sesión, formularios, navegación y comunicación con API.
frontend/style.css	Estilos institucionales y adaptación a pantallas pequeñas.
backend/server.cjs	API, autenticación, roles, validación y recepción de archivos.
backend/db.cjs y migrations	Conexión SQLite, esquema, semilla y bitácora.
backend/certificates.cjs	Emisión, integridad, revocación y reconciliación.
backend/blockchain.cjs	Conexión RPC, wallets, firma y cola de envíos.
backend/pdf.cjs	Diseño del certificado PDF, metadatos y QR.
contracts/AcademicCertRegistry.sol	Reglas ejecutadas en la EVM.
scripts/setup.cjs	Configuración inicial, base y compilación.
scripts/dev.cjs y deploy.cjs	Arranque, restauración local y despliegue.
tests	Pruebas de contrato y de integración HTTP.
runtime	Datos privados generados durante la operación.

5 2 Decisiones de implementación

SQLite se utiliza mediante el módulo incorporado de Node.js, sin Prisma, para reducir dependencias y facilitar la instalación en Windows. bcryptjs proporciona el hash bcrypt sin compilar una extensión nativa. solc se instala como dependencia y Hardhat lo utiliza localmente para no depender de una descarga adicional del compilador.

Las operaciones de escritura a Blockchain se serializan para coordinar el nonce de la wallet institucional. El backend espera la confirmación de cada envío. La red local utiliza cuentas públicas de desarrollo de Hardhat, separadas para administración y emisión. Estas cuentas no deben recibir fondos reales.

5 3 Persistencia entre reinicios

Hardhat mantiene su estado en memoria. El lanzador conserva las transacciones firmadas en chain.jsonl y las reproduce en una red local vacía al reiniciar. Se recuperan los registros y las revocaciones mediante ejecución EVM real. Los hashes de transacción y la dirección se conservan; las fechas y hashes de bloque pueden cambiar.

5 4 Instalación y credenciales de acceso

Requisitos del equipo

Windows 10 u 11, Node.js 24 LTS como entorno recomendado para esta entrega y npm. El proyecto declara un mínimo de Node.js 22.13 y fue probado con Node.js 26.8.1. Visual Studio Code facilita el uso de la terminal. La primera instalación necesita acceso a Internet para descargar dependencias; la demostración local no requiere servicios de pago ni MetaMask.

Instalación desde la carpeta del proyecto

```
cd "C:\Users\Harol\Documents\ChatGPT\Nube 2"
npm install
npm run setup
npm run dev
```

npm install descarga las librerías. setup crea .env si no existe, genera secretos y contraseñas iniciales, crea SQLite y compila el contrato. dev inicia Hardhat, restaura transacciones, despliega cuando corresponde y arranca Express y Vite. Abrir http://localhost:5173 y mantener la terminal activa.

Credenciales iniciales de esta instalación

Rol	Correo de acceso	Contraseña inicial
Administrador	admin@umsa.local	`qIZ9jE7bbNB_1Hxf`
Emisor	emisor@umsa.local	`SrQIuUSeF6hP90q5`

El verificador público entra en http://localhost:5173/verificar sin usuario ni contraseña. Las credenciales anteriores corresponden a la instalación local entregada. En una instalación nueva, setup genera otras contraseñas; deben consultarse en runtime/ACCESO_LOCAL.md o en ADMIN_PASSWORD e ISSUER_PASSWORD de .env.

Este informe incorpora las credenciales porque forma parte del manual de operación del prototipo. Debe conservarse como copia de uso del equipo. No contiene el secreto JWT ni claves privadas de wallets.

Cómo detener y volver a iniciar

Pulsar Ctrl+C en la terminal que ejecuta dev. Para iniciar otra vez, ubicarse en la misma carpeta y ejecutar npm run dev. No es necesario repetir npm install ni setup en cada demostración y no se debe iniciar una segunda instancia sobre los mismos puertos.

5 5 Configuración servicios y acceso por celular

Direcciones y puertos

Servicio	Dirección o puerto	Función
Interfaz	http://localhost:5173	Panel y portal de verificación
API	http://localhost:3001/api	Operaciones y autenticación
Salud	http://localhost:3001/api/health	Estado de API, SQLite y Blockchain
Hardhat RPC	http://127.0.0.1:8545	Comunicación EVM del backend

Variables principales de entorno

PORT define el puerto de Express; HOST controla la escucha de Vite; PUBLIC_URL se incorpora en los QR; RPC_URL y CHAIN_ID seleccionan la red. JWT_SECRET protege las sesiones. CONTRACT_ADDRESS permite usar un contrato existente; en el flujo local se guarda la dirección desplegada en runtime/deployment.json. DATA_DIR puede separar los datos de otro entorno.

```
PORT=3001
HOST=127.0.0.1
PUBLIC_URL=http://localhost:5173
RPC_URL=http://127.0.0.1:8545
CHAIN_ID=31337
```

Acceso desde un teléfono en la misma red

1. Conectar computadora y celular a la misma red Wi-Fi y obtener la IPv4 del equipo con ipconfig.

2. Detener el sistema. Cambiar HOST a 0.0.0.0 y PUBLIC_URL a una URL como http://192.168.1.50:5173, usando la IP real de la computadora.

3. Iniciar otra vez con npm run dev y, si Windows lo solicita, permitir 5173 para la red privada.

4. Emitir un certificado nuevo y escanear su QR con la cámara del teléfono.

Los PDF anteriores conservan la URL con la que fueron generados. El navegador del teléfono utiliza el proxy de Vite; la API y el RPC continúan escuchando en loopback. El QR se comprobó mediante decodificación automática; la prueba con una cámara física queda pendiente.

Comandos de comprobación

```
npm test
npm run build
```

test verifica contrato e integración en puertos aislados. build crea dist para una distribución del frontend; esa carpeta necesita una API y un proxy /api para operar como sistema completo.

5 6 Manual del emisor institucional

Inicio y cierre de sesión

Entrar a la dirección local y escribir el correo y la contraseña del emisor. Al iniciar se muestra el panel con sus certificados. La sesión dura una hora; al vencer, debe ingresar de nuevo. El botón de salida de la barra superior elimina el token del navegador y regresa al acceso institucional.

Registro y edición de estudiantes

1. Abrir Estudiantes y pulsar Registrar estudiante.

2. Completar nombres, apellidos, documento de identidad y carrera. El correo es opcional.

3. Guardar y comprobar que el estudiante aparezca en la tabla. Un documento repetido produce un mensaje de conflicto.

4. Utilizar el buscador para localizar al estudiante. Pulsar Editar para corregir los datos antes de una nueva emisión.

La modificación del estudiante afecta futuras emisiones; no reescribe los certificados previamente registrados. Para corregir un certificado emitido se debe revocar el anterior y generar uno nuevo.

Emisión individual

1. Abrir Emitir certificado y seleccionar al estudiante registrado.

2. Elegir el tipo de certificado e indicar fecha, reconocimiento e institución emisora.

3. Pulsar Emitir y registrar en Blockchain una sola vez y esperar la notificación.

4. Revisar el detalle: debe indicar Emitido y mostrar el hash de los datos, el hash del PDF y la transacción.

5. Pulsar Descargar PDF. El archivo contiene nombre, información académica, código único, QR y la leyenda de prototipo.

Consulta del historial de certificados

En Certificados se puede buscar por estudiante o código y filtrar por fecha o estado. Ver detalle abre la información, el historial de transacciones y las opciones de descarga y verificación. El emisor ve sus propios certificados; el administrador puede consultar todos.

Si una emisión termina en error, no debe presentarse como emitida. El administrador puede reconciliar el registro con la cadena. Si nunca fue registrado, deberá revisarse la causa antes de realizar una nueva emisión.

5 7 Verificación pública y revocación

Verificación por QR o código

El QR es un enlace al portal público con el código del documento. Abrirlo inicia automáticamente la consulta. También se puede entrar a /verificar, introducir el código completo y pulsar Verificar certificado. La respuesta consulta el estado actual del contrato, por lo que un QR impreso antes de la revocación mostrará el nuevo estado cuando vuelva a consultarse.

Verificación del archivo PDF

Seleccionar el archivo original en el portal y pulsar Comprobar integridad del PDF. El servidor lee su UUID, calcula SHA-256 de todos los bytes y lo compara con la huella registrada. La consulta por código acredita el estado del registro; la carga del PDF comprueba además que ese archivo específico coincida.

Resultado	Interpretación
CERTIFICADO VÁLIDO	Existe en cadena, sus huellas coinciden y no está revocado.
CERTIFICADO REVOCADO	Existe un registro, pero su vigencia fue anulada.
CERTIFICADO NO ENCONTRADO	No se encontró un registro verificable para el código o archivo.
DOCUMENTO MODIFICADO O NO COINCIDENTE	La huella o estructura del documento no corresponde al registro esperado.
ERROR DE CONEXIÓN CON BLOCKCHAIN	No fue posible consultar la red; no se confirma validez.
EMISIÓN PENDIENTE	El registro requiere completar o recuperar su estado local.

Revocación desde el panel

1. Localizar un certificado emitido y abrir su detalle.

2. Pulsar Revocar certificado e introducir una justificación de al menos cinco caracteres.

3. Confirmar la operación y esperar el registro en cadena.

4. Comprobar el estado Revocado y consultar de nuevo el código o PDF en el portal público.

La revocación es definitiva. No elimina el PDF ni el registro; conserva la explicación y las transacciones. Solo el emisor propietario o un administrador de la aplicación puede solicitarla. Si se necesita una nueva versión del certificado, se emite otro documento con un nuevo identificador y salt.

5 8 Administración lotes y mantenimiento

Gestión de usuarios y bitácora

El administrador abre Usuarios y acceso para crear una cuenta, asignar ADMIN o EMISOR y establecer una contraseña de al menos diez caracteres. La cuenta queda autorizada y puede desactivarse posteriormente. Desactivar a un usuario bloquea las siguientes solicitudes incluso si su JWT aún no venció. La aplicación impide desactivar la cuenta propia desde esa opción.

Historial de operaciones permite consultar y buscar accesos, altas y cambios de usuarios, emisiones, errores, revocaciones y reconciliaciones. La bitácora de SQLite complementa las transacciones de Blockchain: conserva información de operación que no debe publicarse en la cadena.

Emisión por lote

En Emitir certificado se despliega Emisión por lote CSV. Se descarga la plantilla, se prepara un archivo con las columnas documento, tipo, grado, fecha e institucion y se sube al sistema. Los estudiantes deben existir previamente. Se admiten hasta treinta filas por solicitud.

El backend procesa las filas secuencialmente y devuelve el resultado individual. Una fila inválida no convierte en válidas las demás ni oculta errores. Este procesamiento durante la solicitud simplifica la cola en segundo plano prevista en el informe original.

Recuperación y reconciliación

Desde Certificados, el administrador puede pulsar Reconciliar. El sistema consulta el contrato y comprueba la integridad del PDF antes de actualizar un estado local interrumpido. También intenta reconciliar al iniciar la API. La operación no reconstruye un PDF perdido ni registra automáticamente una emisión que nunca llegó a la cadena.

Copias de respaldo

1. Detener el sistema con Ctrl+C.

2. Copiar runtime completa y .env a una ubicación de respaldo del equipo.

3. Conservar juntos academic.sqlite, pdf, deployment.json y chain.jsonl.

4. Para restaurar, recuperar el conjunto coherente en la misma estructura e iniciar con npm run dev.

El journal registra únicamente transacciones firmadas enviadas por la aplicación. Una operación manual enviada desde otra herramienta no queda incluida automáticamente. El respaldo debe resguardar las credenciales junto con los datos privados, y no debe publicarse en el repositorio.

Capítulo VI Pruebas y resultados de validación

6 1 Estrategia aplicada

La validación combina pruebas del contrato, integración HTTP contra una red Hardhat real y uso del navegador. La suite de integración utiliza una API en el puerto 13001, un nodo en 18545 y una carpeta independiente en test-runtime. Estas pruebas no alteran los certificados de la demostración.

La última ejecución documentada del 22 de octubre de 2026 completó ocho pruebas del contrato y quince casos de integración, sin fallos. El runner de Node informa dieciséis tests porque cuenta también el contenedor del flujo integral. El archivo docs/pruebas-ejecucion.txt conserva la salida completa.

6 2 Pruebas del contrato inteligente

Caso	Comprobación	Resultado
1	Registro de ambos hashes y evento de emisión	Aprobado
2	Rechazo de emisor no autorizado y hash vacío	Aprobado
3	Rechazo de emisión duplicada	Aprobado
4	Hash inexistente devuelve ausencia de registro	Aprobado
5	Revocación del emisor, evento y conservación	Aprobado
6	Otro emisor no puede revocar un registro ajeno	Aprobado
7	Administrador revoca y retira autorización	Aprobado
8	Rechazo de revocación inexistente y gestión ajena	Aprobado

6 3 Compilación y revisión de dependencias

La preparación compiló seis archivos Solidity para EVM Paris. La compilación de producción del frontend con Vite 6.4.5 procesó 1590 módulos y generó dist sin errores. La consulta npm audit --omit=dev no reportó vulnerabilidades en dependencias de producción en esa revisión.

La auditoría completa mantiene avisos transitivos en herramientas de desarrollo de Hardhat y Tailwind. Esta comprobación de paquetes no equivale a una auditoría externa del contrato o del sistema. No se ejecutaron Slither, pruebas de carga, mediciones de costos ni evaluaciones en AWS o Sepolia.

6 4 Casos de integración comprobados

Caso	Comprobación ejecutada	Resultado
1	La API y SQLite responden al chequeo de salud.	Aprobado
2	Login válido y rechazo de contraseña incorrecta.	Aprobado
3	Rutas privadas y restricciones por rol.	Aprobado
4	Alta, edición y rechazo de documento duplicado.	Aprobado
5	Emisión, recibo exitoso y dos hashes en cadena.	Aprobado
6	PDF privado, descarga, metadatos e integridad.	Aprobado
7	Lectura del QR incrustado y consulta de su código.	Aprobado
8	PDF original válido; modificado e inexistente rechazados.	Aprobado
9	Administración, propiedad y desactivación de usuarios.	Aprobado
10	Revocación confirmada y rechazo de repetición.	Aprobado
11	Bitácora y estadísticas desde datos guardados.	Aprobado
12	Recuperación de un estado local interrumpido.	Aprobado
13	CSV con resultados independientes por fila.	Aprobado
14	Caída del nodo produce error, nunca validez aparente.	Aprobado
15	Reinicio de SQLite, API y red conserva registros.	Aprobado

6 4 1 Verificaciones adicionales de interfaz

En el navegador se comprobó el acceso como emisor, el registro de Diego Andrés Flores Condori con documento ficticio DEMO-2026, la emisión y descarga del certificado, su consulta pública como válido y su revocación. Después se emitió un certificado vigente para Ana Lucía Quispe Mamani. El panel muestra dos registros reales: uno emitido y uno revocado.

El portal se revisó con un tamaño de 390 por 844 y una medición de ancho del documento sin desbordamiento horizontal. El PDF se renderizó para comprobar distribución, legibilidad y presencia del QR. La decodificación automatizada del QR fue exitosa; no se realizó un escaneo mediante una cámara física.

6 5 Evidencia del panel institucional

La captura muestra el panel operando con datos guardados en SQLite y transacciones confirmadas en la cadena local. Los contadores reflejan dos certificados: uno válido y uno revocado. La conectividad del nodo se muestra en el menú lateral.

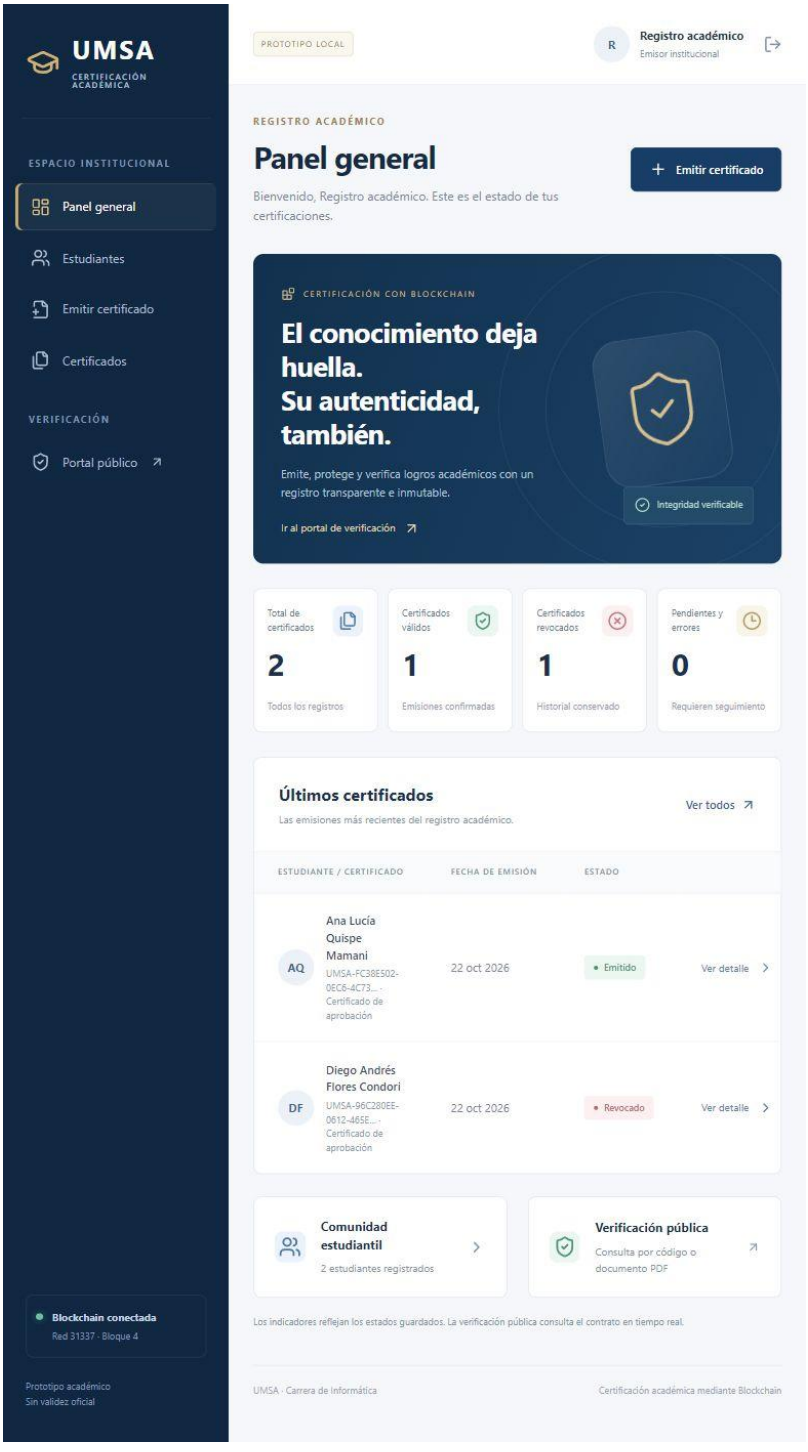


Figura 2 Panel institucional con certificados ficticios registrados

El usuario puede abrir el detalle de cada registro, descargar su documento y acceder al portal de verificación. La apariencia de esta captura corresponde al ancho disponible durante la revisión; la aplicación adapta su distribución a otras pantallas.

6 6 Evidencia del certificado PDF

El documento generado incluye la institución, la carrera, el nombre del estudiante, el reconocimiento, la fecha, el código único y un QR que abre la verificación. No utiliza una firma institucional real y presenta expresamente su condición de prototipo.


UNIVERSIDAD MAYOR DE SAN ANDRÉS

FACULTAD DE CIENCIAS PURAS Y NATURALES · CARRERA DE INFORMÁTICA

CERTIFICADO ACADÉMICO

Certificado de aprobación

Se certifica que

Diego Andrés Flores Condori

Aprobación de Computación en la Nube

Carrera: Informática

Institución: Universidad Mayor de San Andrés

Fecha de emisión: 2026-10-22

Código: UMSA-96C280EE-0612-465E-A6C1-74757FF748F8

Documento verificable digitalmente mediante el código QR.

Prototipo académico - Sin validez oficial

Figura 3 Certificado de prueba emitido y posteriormente revocado

Qué demuestra la comprobación de integridad

El archivo descargado coincide con el hash registrado antes de su entrega. La prueba automatizada añade bytes al PDF y vuelve a cargarlo; el verificador devuelve documento modificado o no coincidente. Esto comprueba el archivo completo, incluso cuando una alteración no produce un cambio evidente en la página visible.

Qué demuestra la revocación

Revocar cambia la vigencia del registro del contrato. El archivo histórico no se borra ni se vuelve a generar, por lo que su hash sigue siendo el mismo. Al consultar ese archivo o su código después de la revocación, el resultado es revocado. La integridad del archivo y la vigencia del certificado son comprobaciones relacionadas, pero distintas.

6 7 Registros de la demostración y trazabilidad

La evidencia siguiente corresponde a la validación local ejecutada. Los titulares y documentos utilizados son ficticios. Las consultas finales devolvieron VALIDO para Ana y REVOCADO para Diego.

Titular	Código del certificado	Estado
Ana Lucía Quispe Mamani	UMSA-FC38E502-0EC6-4C73-9C19-91C7EB932349	Válido
Diego Andrés Flores Condori	UMSA-96C280EE-0612-465E-A6C1-74757FF748F8	Revocado

Red y contrato desplegado

Hardhat local con chain ID 31337. La comprobación de salud observó el bloque 4 y la siguiente dirección del contrato AcademicCertRegistry.

`0x5FbDB2315678afecb367f032d93F642f64180aa3`

Transacciones confirmadas

Emisión del certificado de Ana

`0xb3837aab1b04540b1af01b8ccd955eec72a7b7e92766b7d11ad98bc9df18371c`

Emisión del certificado de Diego

`0x278141cb9d22f22528d049758a2e2e5a2ce8e43efa49691621466aaf203f46cf`

Revocación del certificado de Diego

`0x652ba9b3ccb017d3dcbff8680b5934bec1098a869c4212a4128d5fdd48be20a9`

Alcance temporal de la evidencia

Los estados anteriores describen la validación realizada el 22 de octubre de 2026. Las consultas posteriores dependen de que se conserve el directorio de datos y se inicie la red correspondiente. El bloque observado es una evidencia puntual y no un valor fijo de funcionamiento.

Las pruebas de recuperación detuvieron la API y el nodo, reabrieron SQLite y reprodujeron el journal firmado en una nueva instancia de Hardhat. La consulta del certificado revocado y la presencia del estudiante se conservaron. En la demostración también se reinició el lanzador completo y se recuperaron sus transacciones.

Capítulo VII Conclusiones y evolución del sistema

7 1 Conclusiones de la implementación

El prototipo permite demostrar el ciclo completo de un certificado: datos académicos, documento, registro criptográfico, verificación y revocación. Las operaciones principales se ejecutan sobre persistencia real y un contrato en una EVM local. La verificación no depende únicamente de la apariencia del PDF ni de un estado mostrado por la interfaz.

La separación propuesta en el Capítulo IV se conserva: la base y los archivos privados custodian información personal; Blockchain mantiene huellas y vigencia. El registro adicional de pdfHash permite contrastar el archivo original con el contrato. La bitácora y las transacciones facilitan explicar qué ocurrió en cada operación.

7 2 Límites de la versión entregada

- La ejecución validada es local por HTTP, sin disponibilidad pública ni servicios de nube desplegados.
- Los documentos tienen fines académicos y carecen de validez oficial.
- SQLite y EVM no comparten una transacción atómica; la recuperación se resuelve con estados y reconciliación.
- El journal de Hardhat es una facilidad de desarrollo y no sustituye una red persistente de producción.
- La carga CSV es secuencial, con hasta treinta filas por solicitud.
- No se probaron cámara física, Slither, carga concurrente, costos, AWS, Polygon o Sepolia.

7 3 Evolución coherente con la nube

Una siguiente etapa puede desplegar la aplicación en una máquina virtual, usar HTTPS y un proxy de entrada, trasladar los PDF a S3 privado, adoptar PostgreSQL y gestionar secretos fuera del archivo local. Un proveedor RPC permitiría conectar el backend a Sepolia y después evaluar una red de producción. Esos cambios requieren pruebas adicionales de permisos, disponibilidad, costos y recuperación.

El proyecto ya permite configurar RPC_URL, CHAIN_ID y claves de wallets de prueba para una red externa. La migración debe utilizar un directorio de datos separado y un contrato compatible; en una red externa no se reproduce el journal local. La wallet administrativa debe conservarse fuera del backend después del despliegue.

Anexo A Referencia de la API REST

Todas las rutas tienen el prefijo /api. Las rutas privadas reciben el JWT mediante Authorization con el esquema Bearer. Los permisos se aplican en el servidor y no dependen únicamente de ocultar botones en la interfaz.

Método y ruta	Acceso o finalidad
POST /auth/login	Público; autentica y entrega un JWT.
GET /health	Público; estado de API, base y cadena.
GET /verify/:codigo	Público; consulta el certificado.
POST /verify/file	Público; archivo PDF de hasta 10 MB.
GET /me	Sesión; datos del usuario activo.
GET /dashboard	Sesión; estadísticas y últimos registros.
GET y POST /students	Sesión; buscar y registrar estudiantes.
PUT /students/:id	Sesión; editar un estudiante.
GET y POST /certificates	Sesión; consultar y emitir certificados.
POST /certificates/batch	Sesión; archivo CSV con hasta treinta filas.
GET /certificates/:id	Propietario o ADMIN; detalle e historial.
GET /certificates/:id/pdf	Propietario o ADMIN; descarga autorizada.
POST /certificates/:id/revoke	Propietario o ADMIN; justificación y revocación.
GET y POST /users	ADMIN; listado y creación de usuarios.
PATCH /users/:id	ADMIN; autorizar o desactivar acceso.
GET /audit	ADMIN; historial de acciones.
POST /reconcile	ADMIN; contrastar estados con la cadena.

Respuestas relevantes

La API distingue errores de validación, autenticación, autorización, duplicados y disponibilidad. El frontend presenta mensajes de éxito y error y deshabilita las acciones mientras espera operaciones. El portal utiliza resultados explícitos de verificación y no sustituye un fallo del nodo por un estado válido guardado en memoria.

Anexo B Solución de problemas y exposición

Problemas frecuentes

Situación	Acción recomendada
No inicia por puerto ocupado	Cerrar la instancia anterior. Evitar ejecutar simultáneamente dos lanzadores o una red manual en 8545.
No reconoce node o npm	Instalar Node.js y abrir otra terminal de VS Code. Comprobar node --version y npm --version.
Credenciales rechazadas	Consultar las de esta instalación en runtime/ACCESO_LOCAL.md. Revisar que el usuario siga activo.
Blockchain no disponible	Comprobar que npm run dev mantenga el nodo activo y que RPC_URL y CHAIN_ID sean correctos.
Emisión en error	Revisar la salida de la API y usar Reconciliar como administrador cuando exista evidencia en cadena.
El QR no abre en el teléfono	Configurar PUBLIC_URL con la IPv4 de la computadora y emitir un PDF nuevo. Revisar la red y el puerto 5173.
PDF modificado o perdido	Usar el archivo original descargado. Si falta el privado, restaurar el respaldo coherente.
El JWT venció	Volver a iniciar sesión. La duración configurada es de una hora.

Guion sugerido para la exposición

1. Mostrar el panel y explicar que sus indicadores proceden de SQLite.
2. Ingresar como emisor y registrar un estudiante ficticio con documento único.
3. Emitir el certificado y mostrar ambos hashes y la transacción confirmada.
4. Descargar el PDF y señalar el QR y la leyenda Sin validez oficial.
5. Consultar el código o el QR y demostrar el estado válido.
6. Cargar el PDF original y explicar la comprobación de integridad.
7. Revocar con un motivo y volver a verificar para mostrar el cambio.
8. Cerrar explicando la separación de datos privados y hashes, y la futura adaptación a nube.

Anexo C Glosario y fuentes del proyecto

Glosario de términos

Término	Significado en este proyecto
API REST	Interfaz HTTP que conecta la aplicación React con el backend.
Blockchain	Registro ejecutado por una EVM que conserva hashes y revocaciones.
EVM	Máquina virtual que ejecuta el contrato Solidity.
Hash SHA-256	Huella de 256 bits calculada a partir de datos o bytes del PDF.
Salt	Valor aleatorio privado incorporado al hash de los datos.
UUID	Identificador único utilizado para el certificado.
JWT	Token firmado que representa la sesión del usuario.
Wallet	Cuenta que firma las transacciones del contrato.
RPC	Interfaz mediante la que ethers consulta y envía transacciones.
On-chain y off-chain	Información dentro de la cadena y fuera de ella.
Revocación	Anulación definitiva de la vigencia, sin borrar el registro.
Reconciliación	Comprobación del contrato y del archivo para recuperar un estado local.

Fuentes documentales y técnicas

1. Informe académico del equipo: Documento sin título (1).docx. Universidad Mayor de San Andrés, Computación en la Nube, 2026. Base de organización y diseño, especialmente el Capítulo IV.

2. Código fuente implementado, package.json y package-lock.json. Contrato AcademicCertRegistry.sol, backend, frontend y scripts del proyecto.

3. README.md, runtime/ACCESO_LOCAL.md, docs/VALIDACION.md y docs/pruebas-ejecucion.txt. Instalación, credenciales y resultados de la ejecución documentada del 22 de octubre de 2026.

4. Diagramas de arquitectura, datos, emisión y verificación conservados en docs/diagramas; capturas y render del PDF en docs/evidencias.

Las cifras de validación de este documento corresponden a pruebas ejecutadas del prototipo. No se trasladan al resultado local las afirmaciones del informe original sobre costos, tiempos de Sepolia, carga de usuarios o servicios cloud que no fueron medidas en esta implementación.